Sesión 13: Navegación y Enrutamiento (Navigator 1.0 vs. Navigator 2.0 Declarativo, Manejo de Rutas Nombradas y Paso de Argumentos)

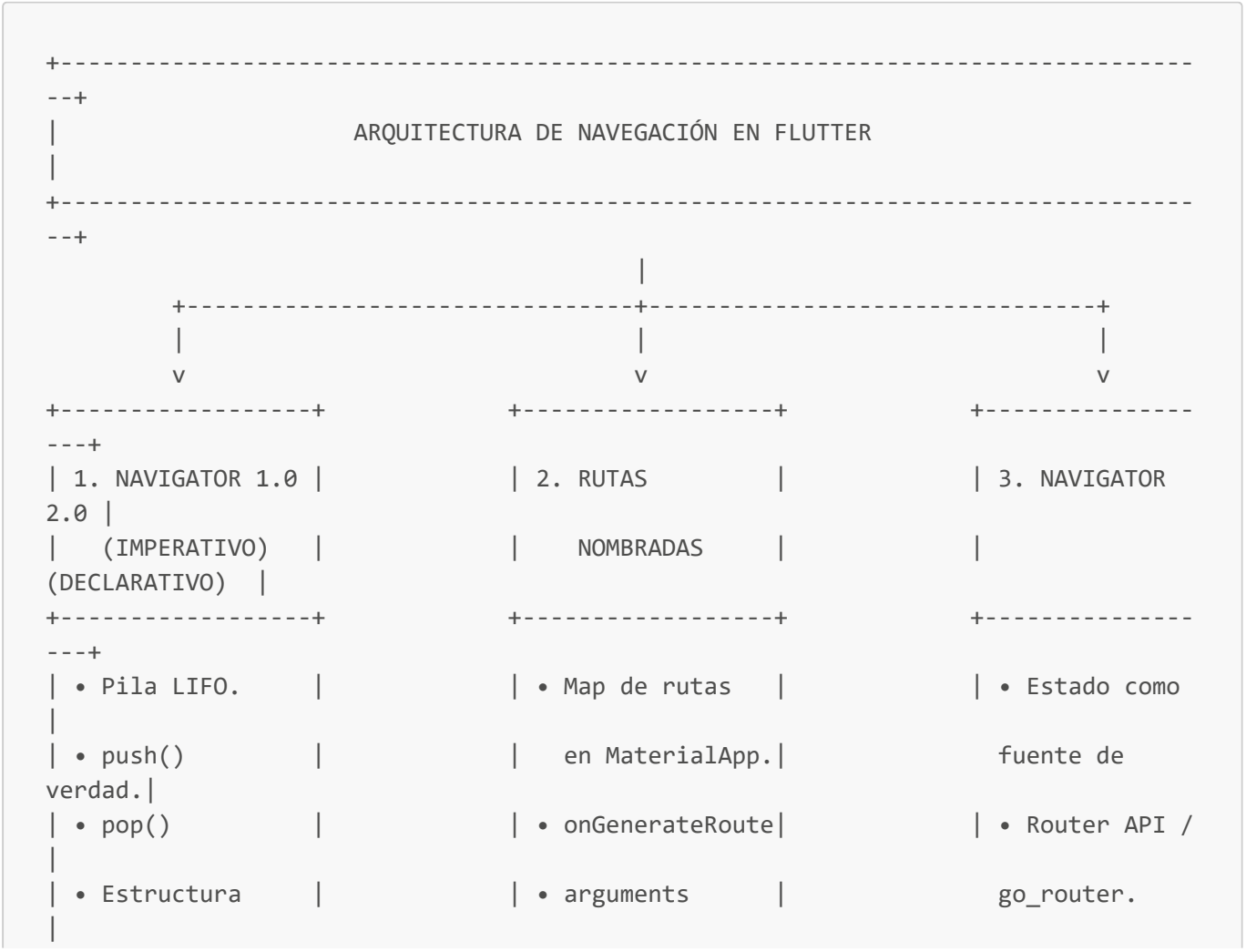
Introducción

La navegación es la columna vertebral de la experiencia de usuario en cualquier aplicación móvil o web. Administrar cómo el usuario se desplaza entre diferentes pantallas, cómo se pasan datos de una vista a otra y cómo se responde al botón "Atrás" del sistema exige una estructura de enrutamiento clara, escalable y mantenible.

En esta sesión profundizaremos en el sistema de navegación de Flutter. Analizaremos las diferencias conceptuales entre el paradigma imperativo (**Navigator 1.0**) y el paradigma declarativo (**Navigator 2.0** / Router API), la configuración de rutas nombradas (`initialRoute`, `routes`, `onGenerateRoute`) y las técnicas robustas para la transferencia de argumentos entre pantallas.

Conceptos y Definiciones

El sistema de navegación en Flutter gestiona la pila de pantallas (*route stack*) en la que cada vista se superpone sobre la anterior.



```
|   simple.        |             |   dinámicos.     |             | • Deeplinking.
|
+------------------+             +------------------+             +---------------
---+
```

1. Navigator 1.0 vs. Navigator 2.0

- **Navigator 1.0 (Paradigma Imperativo):**
- Modifica la pila de navegación directamente ejecutando comandos como `Navigator.push()` para apilar una pantalla o `Navigator.pop()` para desapilarla.
- Es ideal para aplicaciones móviles con flujos secuenciales sencillos.
- *Desventaja:* Dificulta la gestión de enlaces profundos (*deep linking*) en web o flujos complejos donde el estado de la aplicación debe determinar la pila de pantallas.
- **Navigator 2.0 (Paradigma Declarativo / Router API):**
- La interfaz de usuario refleja directamente el estado de la aplicación (`Pages`). Si el estado cambia, la pila de navegación se re-renderiza automáticamente.
- Permite soportar URLs complejas en la web, sincronizar el botón de retroceso del navegador y gestionar rutas anidadas.
- *Práctica en la industria:* Aunque la API nativa de Navigator 2.0 es muy verbosa (`Router`, `Delegate`, `RouteInformationParser`), se suele implementar en producción mediante la librería oficial recomendada por el equipo de Flutter: **`go_router`**.

---

2. Rutas Nombradas y Mapa de Enrutamiento

Para evitar acoplar directamente las clases de las pantallas al navegar, Flutter permite asociar cadenas de texto (*strings*) con identificadores de rutas (ej. `'/'`, `'/detalle'`):

- **`initialRoute`:** Define la ruta que se desplegará al arrancar la aplicación.
- **`routes`:** Mapa de tipo `Map<String, WidgetBuilder>` que asocia una clave de texto con el creador del widget correspondiente.
- **`onGenerateRoute`:** Callback dinámico que intercepta peticiones de navegación a rutas no definidas estáticamente en `routes`. Es el lugar óptimo para extraer argumentos, realizar validaciones de acceso o aplicar transiciones personalizadas (`PageRouteBuilder`).

---

3. Paso y Extracción de Argumentos

Para transferir datos entre pantallas existen dos enfoques principales:

1. **Paso Directo vía Constructor:** Se pasan las variables directamente al instanciar el widget en un `MaterialPageRoute`.

2. **Paso de Argumentos Nombrados (`ModalRoute`):** Se envían datos como un objeto dinámico a través del parámetro `arguments` en `Navigator.pushNamed()` y se leen en la pantalla de destino utilizando `ModalRoute.of(context)!.settings.arguments`.

Ejemplo Completo con Código Fuente: Navegación por Rutas Nombradas, Captura de Argumentos y Retorno de Datos

El siguiente código implementa una aplicación con tres pantallas que demuestra el flujo completo de navegación imperativa estructurada: catálogo de elementos, detalle con argumentos e intercepción de retorno de datos.

Código Fuente Explicado (`lib/main.dart`)

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const DemoNavegacionApp());
}

/// Modelo de datos para transferencia de información entre pantallas
class Curso {
  final String id;
  final String nombre;
  final String instructor;

  const Curso({
    required this.id,
    required this.nombre,
    required this.instructor,
  });
}

/// Widget Raíz con Configuración Centralizada de Rutas
class DemoNavegacionApp extends StatelessWidget {
  const DemoNavegacionApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Navegación y Enrutamiento',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      // Definición de la ruta inicial
      initialRoute: '/',

      // Mapa de Rutas Nombradas Estáticas
```

```
      routes: {
        '/': (context) => const PantallaCatalogo(),
      },

      // Administrador Dinámico de Rutas (Manejo de Argumentos)
      onGenerateRoute: (settings) {
        if (settings.name == '/detalle') {
          // Extracción segura del argumento
          final curso = settings.arguments as Curso;
          return MaterialPageRoute(
            builder: (context) => PantallaDetalleCurso(curso: curso),
          );
        }
        // Ruta de respaldo (404)
        return MaterialPageRoute(
          builder: (context) => const PantallaNoEncontrada(),
        );
      },
    );
  }
}

/// 1. PANTALLA PRINCIPAL: CATÁLOGO DE CURSOS
class PantallaCatalogo extends StatefulWidget {
  const PantallaCatalogo({super.key});

  @override
  State<PantallaCatalogo> createState() => _PantallaCatalogoState();
}

class _PantallaCatalogoState extends State<PantallaCatalogo> {
  final List<Curso> _listaCursos = const [
    Curso(
      id: 'DART-01',
      nombre: 'Fundamentos de Dart 3',
      instructor: 'Prof. Ozzy',
    ),
    Curso(
      id: 'FLUT-02',
      nombre: 'Arquitectura de Software Móvil',
      instructor: 'Prof. Ozzy',
    ),
    Curso(
      id: 'SEC-03',
      nombre: 'Ciberseguridad y OWASP',
      instructor: 'Prof. Ozzy',
    ),
  ];

  String _ultimoMensajeRespuesta = 'Sin acciones recientes.';

  @override
  Widget build(BuildContext context) {
    return Scaffold(
```

```
      appBar: AppBar(
        title: const Text('Catálogo de Cursos - ESCOM'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
      ),
      body: Column(
        children: [
          Container(
            width: double.infinity,
            padding: const EdgeInsets.all(16.0),
            color: Theme.of(context).colorScheme.surfaceContainerHigh,
            child: Text(
              'Estado de retorno: $_ultimoMensajeRespuesta',
              style: const TextStyle(fontWeight: FontWeight.bold),
            ),
          ),
          Expanded(
            child: ListView.builder(
              itemCount: _listaCursos.length,
              itemBuilder: (context, index) {
                final curso = _listaCursos[index];
                return ListTile(
                  leading: CircleAvatar(
                    child: Text(curso.id.substring(0, 2)),
                  ),
                  title: Text(curso.nombre),
                  subtitle: Text('Instructor: ${curso.instructor}'),
                  trailing: const Icon(Icons.arrow_forward_ios, size: 16),
                  onTap: () async {
                    // Navegación con envío de argumentos y espera de resultado
futuro (Future)
                    final resultado = await Navigator.pushNamed(
                      context,
                      '/detalle',
                      arguments: curso,
                    );

                    if (resultado != null && mounted) {
                      setState(() {
                        _ultimoMensajeRespuesta = resultado.toString();
                      });
                    }
                  },
                );
              },
            ),
          ),
        ],
      ),
    );
  }
}

/// 2. PANTALLA DETALLE: MUESTRA ARGUMENTOS Y RETORNA DATOS
```

```
class PantallaDetalleCurso extends StatelessWidget {
  final Curso curso;

  const PantallaDetalleCurso({
    super.key,
    required this.curso,
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(curso.id),
      ),
      body: Padding(
        padding: const EdgeInsets.all(24.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              curso.nombre,
              style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                    fontWeight: FontWeight.bold,
                  ),
            ),
            const SizedBox(height: 8),
            Text(
              'Instructor a cargo: ${curso.instructor}',
              style: Theme.of(context).textTheme.titleMedium,
            ),
            const Divider(height: 32),
            const Text(
              'Esta pantalla fue cargada extrayendo el objeto "Curso" a través de
los argumentos de la ruta en onGenerateRoute.',
            ),
            const Spacer(),

            // BOTÓN PARA DEVOLVER UN VALOR A LA PANTALLA ANTERIOR
            Row(
              children: [
                Expanded(
                  child: ElevatedButton.icon(
                    onPressed: () {
                      // Cierra la pantalla y devuelve una cadena de texto
                      Navigator.pop(
                        context,
                        'Curso "${curso.nombre}" INSCRITO correctamente.',
                      );
                    },
                    icon: const Icon(Icons.check),
                    label: const Text('Inscribirme'),
                    style: ElevatedButton.styleFrom(
                      backgroundColor: Colors.green.shade700,
                      foregroundColor: Colors.white,
```

```
                    ),
                  ),
                ),
                const SizedBox(width: 12),
                Expanded(
                  child: OutlinedButton.icon(
                    onPressed: () {
                      Navigator.pop(context, 'Acción CANCELADA por el usuario.');
                    },
                    icon: const Icon(Icons.close),
                    label: const Text('Cancelar'),
                  ),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}

/// 3. PANTALLA DE ERROR 404 (FALLBACK)
class PantallaNoEncontrada extends StatelessWidget {
  const PantallaNoEncontrada({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Ruta No Encontrada')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(Icons.error_outline, size: 64, color: Colors.red),
            const SizedBox(height: 16),
            const Text('La pantalla solicitada no existe.'),
            const SizedBox(height: 16),
            ElevatedButton(
              onPressed: () {
                Navigator.pushReplacementNamed(context, '/');
              },
              child: const Text('Volver al Inicio'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Prácticas Guiadas / Laboratorio

Práctica 1: Intercepción de Botón de Retroceso Físico con `PopScope`

1. Envuelve el widget `Scaffold` de la pantalla `PantallaDetalleCurso` con el widget `PopScope`.
2. Configura la propiedad `canPop: false` para bloquear el cierre involuntario de la pantalla cuando el usuario presiona el botón "Atrás" del sistema.
3. Implementa el callback `onPopInvokedWithResult` para desplegar un diálogo de confirmación (`AlertDialog`) que pregunte si el usuario realmente desea salir sin guardar cambios:

```
PopScope(
  canPop: false,
  onPopInvokedWithResult: (didPop, result) async {
    if (didPop) return;
    final confirmar = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('¿Desea salir?'),
        content: const Text('Se perderán las acciones no guardadas.'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('NO'),
          ),
          TextButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('SÍ'),
          ),
        ],
      ),
    );
    if (confirmar == true && context.mounted) {
      Navigator.pop(context, 'Salida confirmada.');
    }
  },
  child: Scaffold(...),
)
```

Práctica 2: Introducción Práctica al Enrutamiento Declarativo con `go_router`

1. Modifica la configuración de dependencias de un proyecto Flutter agregando el paquete oficial `go_router`.
2. Crea una instancia de `GoRouter` especificando rutas mediante la sintaxis declarativa (`GoRoute`):

```
final _router = GoRouter(
  routes: [
    GoRoute(
```

```
        path: '/',
        builder: (context, state) => const PantallaCatalogo(),
      ),
      GoRoute(
        path: '/detalle',
        builder: (context, state) {
          final curso = state.extra as Curso;
          return PantallaDetalleCurso(curso: curso);
        },
      ),
    ],
  );
```

3. Sustituye la propiedad `home` o `routes` en `MaterialApp.router` para asociarle el objeto `_router`.

Resumen y Puntos Clave

- **Navigator 1.0 (Imperativo):** Opera con una pila LIFO (*Last In, First Out*) manipulada mediante `push()`, `pop()`, `pushNamed()` y `pushReplacementNamed()`.
- **Rutas Nombradas:** Permiten independizar el código de la navegación asignando identificadores de texto centralizados en la configuración de `MaterialApp`.
- **`onGenerateRoute`:** Ofrece un control fino e intermediación dinámica para procesar argumentos, validar permisos o construir transiciones a medida antes de instanciar un widget.
- **Retorno de Datos Asíncrono:** La llamada a `Navigator.push()` devuelve un objeto `Future<T>`, lo que permite recibir datos devueltos desde la pantalla de destino mediante `Navigator.pop(context, resultado)`.
- **Navigator 2.0 y `go_router`:** Evoluciona el paradigma hacia una arquitectura basada en estado declarativo, siendo imprescindible para aplicaciones web y manejo de *deep linking*.

Bibliografía y Referencias

1. **Google LLC.** (2024). *Navigation and routing - Flutter Docs*. Recuperado de: https://docs.flutter.dev/ui/navigation
2. **Google LLC.** (2024). *Navigate with named routes - Flutter Cookbooks*. Recuperado de: https://docs.flutter.dev/cookbook/navigation/named-routes
3. **Google LLC.** (2024). *Package go_router - pub.dev*. Recuperado de: https://pub.dev/packages/go_router
4. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.